Combining LLM-Generated and Test-Based Feedback in a MOOC for Programming

Hagit Gabbay†
School of Education
Tel Aviv University
Tel Aviv, Israel
hagitgabbay@mail.tau.ac.il

Anat Cohen
School of Education
Tel Aviv University
Tel Aviv, Israel
Anatco@tauex.tau.ac.il

ABSTRACT

In large-scale programming courses, providing learners with immediate and effective feedback is a significant challenge. This study explores the potential of Large Language Models (LLMs) to generate feedback on code assignments and to address the gaps in Automated Test-based Feedback (ATF) tools commonly employed in programming courses. We applied dedicated metrics in a Massive Open Online Course (MOOC) on programming to assess the correctness of feedback generated by two models, GPT-3.5-turbo and GPT-4, using a reliable ATF as a benchmark. The findings point to effective error detection, yet the feedback is often inaccurate, with GPT-4 outperforming GPT-3.5-turbo. We used insights gained from the prompt practices to develop Gipy, an application for submitting course assignments and obtaining LLM-generated feedback. Learners participating in a field experiment perceived the feedback provided by Gipy as moderately valuable, while at the same time recognizing its potential to complement ATF. Given the learners' critique and their awareness of the limitations of LLM-generated feedback, the studied implementation may be able to take advantage of the best of both ATF and LLMs as feedback resources. Further research is needed to assess the impact of LLM-generated feedback on learning outcomes and explore the capabilities of more advanced models.

CCS CONCEPTS

Social and professional topics~Professional topics~Computing education•Applied computing~Education~Interactive learning environments

KEYWORDS

Automated feedback, MOOC for programming, Programming education, Generative AI, Large Language Models (LLMs)

L@S '24, July 18–20, 2024, Atlanta, GA, USA

ACM ISBN 979-8-4007-0633-2/24/07.
https://doi.org/10.1145/3657604.3662040

ACM Reference Format:
Hagit Gabbay and Anat Cohen. 2024. Combining LLM-Generated and Test-Based Feedback in a MOOC for Programming. *In Proceedings of the Eleventh ACM Conference on Learning @ Scale (L@S '24), July 18-20, 2024, Atlanta, GA, USA*, ACM, New York, NY, USA. 11 pages. https://doi.org/10.1145/3657604.3662040

1 INTRODUCTION

The high demand for programming expertise has led to a growing number of programming learners, many of whom study independently through online courses [6]. Practicing code writing, coupled with immediate feedback on the code's correctness and quality, is crucial to developing programming skills [13]. Consequently, the need for automated systems to provide feedback on coding assignments in large-scale learning has increased, and instructors continuously seek advanced yet resource-efficient methods to provide adaptive feedback and assess various features of the code [34]. Still most of the systems available today offer Automated Test-based Feedback (ATF) [21], which addresses the differences between obtained and expected results rather than being tailored to the submitted code [5, 17]. Despite its accuracy and reliability [36], this feedback focuses solely on the code's correctness, while overlooking quality or efficiency considerations [34].

Generative AI tools, reinforced by LLMs such as the Generative Pre-trained Transformer (GPT), have exhibited remarkable capabilities for generating and interpreting natural language data, as well as for understanding and producing source code [39]. Combined with the growing availability of these models [22], these proficiencies have introduced both opportunities and challenges in a variety of computing education contexts [4, 30, 39], including writing code and generating solutions for code assignments [8, 11, 12], explaining code [25], generating learning resources [7], and automatically fixing bugs [24, 44]. In particular, the ability of LLMs to generate context-specific responses may address the demand for feedback tailored to learners' needs [9, 12].

Several studies have demonstrated the potential of GPT and other LLMs to generate feedback for code assignments (e.g., [23, 28]). Nevertheless, some issues have also emerged, including inconsistency in reproducible results and inadequate error analysis [18, 19, 22, 28]. Furthermore, while LLMs exhibit high abilities in understanding code syntax, their performance in

detecting logical and runtime errors without explicitly running the code, as ATF tools do, remains underexplored. Combining the capabilities of LLMs with the trustworthy feedback provided by established ATF tools may address the shortcomings of both feedback sources, thereby providing learners with tailored, detailed, and precise feedback.

The current study aimed to explore the potential of incorporating LLM-generated feedback within a Massive Open Online Course (MOOC) for programming to provide effective support for novices in solving code assignments, both as a stand-alone tool and a complement to test-based feedback. To that end, we developed Gipy, a user-friendly and easy-to-deploy application that facilitates the submission of code assignment solutions and provides LLM-generated feedback. The following research questions guided our study:

RQ1: To what extent is LLM-generated feedback for programming assignments correct and accurate? Specific areas of focus include correct identification of various error types, precise feedback formulation, and model consistency.

RQ2: How do learners perceive the usability of the feedback generated by LLM?

RQ3: How do learners perceive the contribution of LLM-generated feedback to solving code assignments, both as standalone support and to complement feedback provided by an ATF system?

To address these questions, we implemented Gipy in a MOOC for learning Python as a supplementary tool alongside the ATF system used in the course. We proceeded in two ways: Offline, we evaluated the correctness and accuracy of the feedback on learners' solutions for the code assignments that was generated by two models: GPT-3.5-turbo and GPT-4. Additionally, we conducted a field experiment. This entailed introducing Gipy to learners enrolled in the course so that they could experience the GPT-generated feedback as they completed the assignment and submitted their solutions. The learners' perceptions were gathered through queries appearing next to the feedback and via a questionnaire at the end of the study. This study adds to the growing body of research exploring the capacity of LLMs to provide helpful and personalized feedback for programming learners. In addition to evaluating the feedback generated for various error types, the study also sheds light on learners' perceptions of LLM-generated feedback that complements the reliable, albeit limited, test-based feedback. Our findings offer educators insights into the practicality of integrating cost-effective LLM-generated feedback into large-scale programming courses, especially those already incorporating an ATF system.

2 RELATED WORK

2.1 Automated feedback in programming courses

Various factors influence the effectiveness of feedback provided by ATF systems in programming courses, such as the content and focus of the feedback [15, 33]. A comparison of the impact of different types of feedback [17] suggests that elaborated feedback is more effective than feedback that merely provides an indication of correctness. Recent studies highlighted the advantage of next-step hints tailored to the submitted code over feedback offering generic hints [17, 32]. [38] formulates several concepts for ensuring the quality of ATF systems, such as flexibility in accommodating variations in students' programs and evaluation of partially correct code, as well as code quality considerations and adherence to coding standards. The readability of feedback messages is another issue shown to have a positive effect on feedback impact, especially in the context of syntax errors [10]. In this regard, feedback provided by ATF systems is not satisfactory [21, 34].

Static analysis, which uses textual analysis rather than code execution to assess the submitted code, harnesses methods such as Natural Language Processing and abstract representation. This approach, which is gaining increasing attention in automated feedback research, directly examines the source code, allowing for tailored feedback and the possibility of addressing code quality, syntax errors, and structural requirements [31, 34]. In the current study we explore the potential of recent advancements in LLMs to take this text-based feedback generation approach a step forward.

2.2 LLM-generated feedback for code assignments

Several recent studies have examined the potential of LLM to provide feedback on code assignments and assist learners, with emphasis on precise identification of errors in the code and accuracy of the generated feedback [2, 18, 22, 37]. When assessing incorrect code, the examined models often detected at least one issue correctly, but they did not excel at finding all the errors and they exhibited high rates of False Positives (identifying incorrect code as correct) [18, 22]. The provided feedback typically explained error causes and suggested improved code and actionable insights [22, 26]. Yet in many cases the feedback was incomplete or contained misleading information [1, 22]. Studies that compare multiple models revealed capability enhancement that correlated with the technological progression of the models [18, 29].

Much of the research on LLM-generated feedback is based on expert evaluations or lab settings (e.g. [19, 22, 26, 29, 40]). Two exceptions are notable: In the study by [35], students who utilized LLM-generated hints in addition to regular ATF outperformed a control group using standard feedback only. Moreover, the students in the experimental group showed decreased dependence on ATF and positively rated the usefulness of LLM-generated hints. Similarly, students in a data science course favoured LLM-generated feedback due to its accessibility and error-resolution assistance [28]. Instructors appreciated the straightforward deployment of the LLM-generated feedback and the fact that it complemented rather than replaced the assistance they gave their students. Our work contributes to this stream of research by exploring the potential of LLM-generated feedback to assist learners within the context of independent programming learning, specifically in a MOOC. To mitigate the issue of

misleading information, we offer LLM-generated feedback as a complementary tool, in conjunction with an ATF system.

3 METHOD

Two experiments were conducted in this study: The first addressed RQ1 by evaluating the correctness and accuracy of LLM-generated feedback, with the ATF serving as a benchmark. The second addressed RQ2 and RQ3 via Gipy, an application that enabled learners to receive LLM-generated feedback alongside feedback from the ATF integrated in the course. In this field experiment, two sets of queries assessed the usability and perceived value of the feedback provided by Gipy.

This study primarily evaluates feedback from the GPT-3.5-turbo model, chosen for its widespread availability and free access. The Gipy application was thus developed using this model. Nevertheless, with the release of GPT-4 during our research, we extended the evaluation of feedback correctness and accuracy (RQ1) to encompass this advanced model as well. In doing so, we aimed to assess the improvements in the newer model's feedback generation capabilities and to evaluate whether and how such advancements may influence the viability of incorporating LLM-generated feedback in programming courses in the future.

3.1 Course Context and the ATF System

Our research focuses on a self-paced MOOC for learning Python programming on an Edx-based platform, aimed at beginners with no prior knowledge. The course comprises nine learning units that cover the basics of Python programming and more advanced topics such as functions, data structures, and file operations. To provide learners with hands-on experience, forty-seven code-writing assignments are offered, with solutions ranging from a few to several dozen lines of code. For some assignments a function is the solution, while others require a complete program.

Learners can obtain feedback on their submissions via an ATF system, incorporated into the course as an external tool. The system runs the submitted code against a predetermined, assignment-tailored set of test scenarios and provides immediate feedback. The detected errors include syntax errors (SYE), incorrect implementation of instructions (INE), exception errors that prevent the code from being executed (EXE), and test-failed errors (TFE) in which the results do not match the expected ones. The feedback consists of a grade, an indication of correctness, and a text message. In the case of SYEs, the compiler message is conveyed as is. For other error types, the message includes further details, such as the expected result and general hints toward the solution. Although adapted to each individual assignment, test scenario, and error type, these textual messages are predefined and do not refer to the errors in the particular submitted code. All submissions and feedback are recorded in the system's log file.

3.2 Prompting the LLMs for feedback

Given the pivotal role of prompt design and wording in determining the performance of LLMs [20], our first step involved the formulation of a consistent prompt framework, which facilitated the conveyance of assignment-solution pairs to elicit optimal responses within the context of this study. Additionally, through this prompt we sought to "enforce" two pedagogical principles relevant to the generated feedback: refraining from disclosing the correct solution and avoiding inundating learners with information not directly related to the instructions of the assignment [9]. Because the learners are not native English speakers, precise translation and articulation of the assignments also constituted a challenge.

Following the guidelines for prompt engineering as outlined in recent literature [16, 20, 35, 43], we crafted an initial version of the prompt, consists of several components:

- **Role and context:** The role of the model is that of an assistant within a programming course setting.
- **Variables**: The input includes a programming assignment and its corresponding solution, which the model is tasked with evaluating.
- **Task description**: The task is framed as a question to guide the model's response direction: "What is the feedback for this solution"?
- **Restrictions**: The model is instructed to base its feedback solely on the assignment's instructions without proposing an alternative or correct solution.

The temperature parameter, which controls randomness in LLM responses, was set at zero to minimize the variability of the model's responses [41].

Initial tests with this prompt revealed variability in the format and detail level of the model's responses. To address this, we enhanced the prompt by including a clear definition of the desired output, specifying the exact phrasing for feedback on whether a solution is correct or incorrect.

```
"You are a helpful assistant in a programming course. The
assignment is:
{assignment}
The solution is:
{solution}
What is the feedback for this solution?
If the solution is correct, write only 'The solution is correct!'.
If not, write 'The solution is incorrect,' and display the errors
(including a line number for each error) and discrepancies
from the instructions in numbered lines.
Do not reveal the full correct solution or parts of it."
```

Figure 1: Structure of the final prompt.

To refine the generated response, an iterative process was then conducted utilizing two sets of solutions, for which the expected feedback was known, as a benchmark: expert solutions for all assignments, and solutions containing specific errors for each assignment. We converged on an optimized prompt (Figure 1) that elicited a response of "The solution is correct" for all iterations with expert solutions and concurrently yielded the highest proportion of accurate feedback for the erroneous ones. We employed this prompt, along with precise articulation of the assignments, for the subsequent phases of our research and for both evaluated versions of GPT.

3.3 LLM-generated Feedback Assessment

To address RQ1, a set of learners' submissions for the course assignments was extracted from the ATF log file of the course cycle scheduled from July 2022 to January 2023. An aggregate of 500 solutions was randomly selected, comprising 24.8% correct solutions (124) and 75.2% incorrect solutions (376) and featuring various types of errors. Table 1 shows the distribution of error types among the incorrect solutions.

Table 1: Error type distribution in incorrect solutions

Error type	N	Percentage (out of N=376 incorrect solutions)
EXE	50	13%
INE	58	15%
SYE	106	28%
TFE	162	43%
Total	376	

We applied the OpenAI API[1] and the established prompt framework to send the code snippets to each of the two models (i.e., GPT-3.5-turbo and GPT-4) for assessment. The obtained feedback was compared to the feedback provided by the ATF system for the same code. The correctness of the generated feedback was evaluated by three metrics:

1. **Error detection score**: Proportion of solutions with matching decisions (correct/incorrect) by both ATF and GPT model. In case the GPT model provided a different decision, we differentiate between a False Positive signifying an incorrect solution identified as correct and a False Negative indicating a correct solution identified as incorrect.
2. **Feedback accuracy rate:** Assessment of feedback message accuracy (i.e., type of detected errors, line numbers in which errors were detected, misconception descriptions). The accuracy rate ranges between 0-100 and is determined by the number of correct comments out of the total comments included in the feedback message. For example, feedback containing five comments, of which three are correct, will yield an accuracy rate of 60.
3. **Prompt compliance:** The percentage of cases in which the prompt instructions are followed (YES / NO).

To account for the inherent inconsistency within LLMs, we ran three iterations with each of the GPT models, utilizing the same dataset. The error detection score for each model was computed as an average across the three iterations. To evaluate accuracy, a Python expert manually examined the feedback message provided for a subset of solutions that were correctly flagged as incorrect. A total of 158 incorrect solutions (52%) were assessed, with various error types (Table 2). To evaluate the model's consistency, we compared the feedback generated for each solution among the three iterations. The temperature in this phase was set at 0.5 to allow for higher variability in the model's responses.

3.4 The Gipy Application

Our goal in developing Gipy was to give learners the opportunity to experience LLM-generated feedback as a complementary tool in solving course assignments. Gipy features an intuitive user interface that allows learners to select an assignment and upload, or directly write, the solution to be assessed (Figure 2). Behind the scenes, the application is pre-loaded with all the course assignments. When a user selects an assignment by number and submits the solution, the application composes the designed prompt framework with the chosen assignment and code and sends it to the GPT-3.5-turbo model via the OpenAI API. The generated feedback is subsequently displayed.

Table 2: Error type distribution for accuracy assessment

Error type	N	Percentage (out of N=158 solutions in the subset)
EXE	32	20%
INE	24	15%
SYE	49	31%
TFE	53	34%
Total	158	

If the submitted code is deemed correct, the message "The solution is correct" appears. Upon detecting an incorrect solution, the application provides feedback (Figure 3, upper section), along with two research-specific queries (discussed in the following section). All user interactions are logged, with each learner identified by a unique user key for tracking individual performance and engagement. Gipy is implemented as a web application for easy access on different devices.

[1] OpenAI API reference

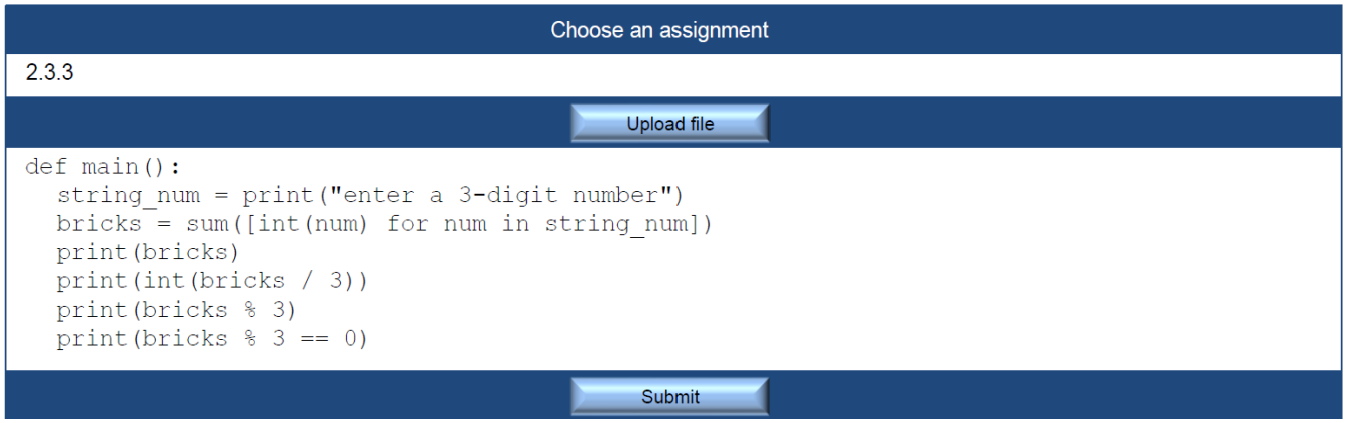

Figure 2: Gipy interface - solution uploading screen.

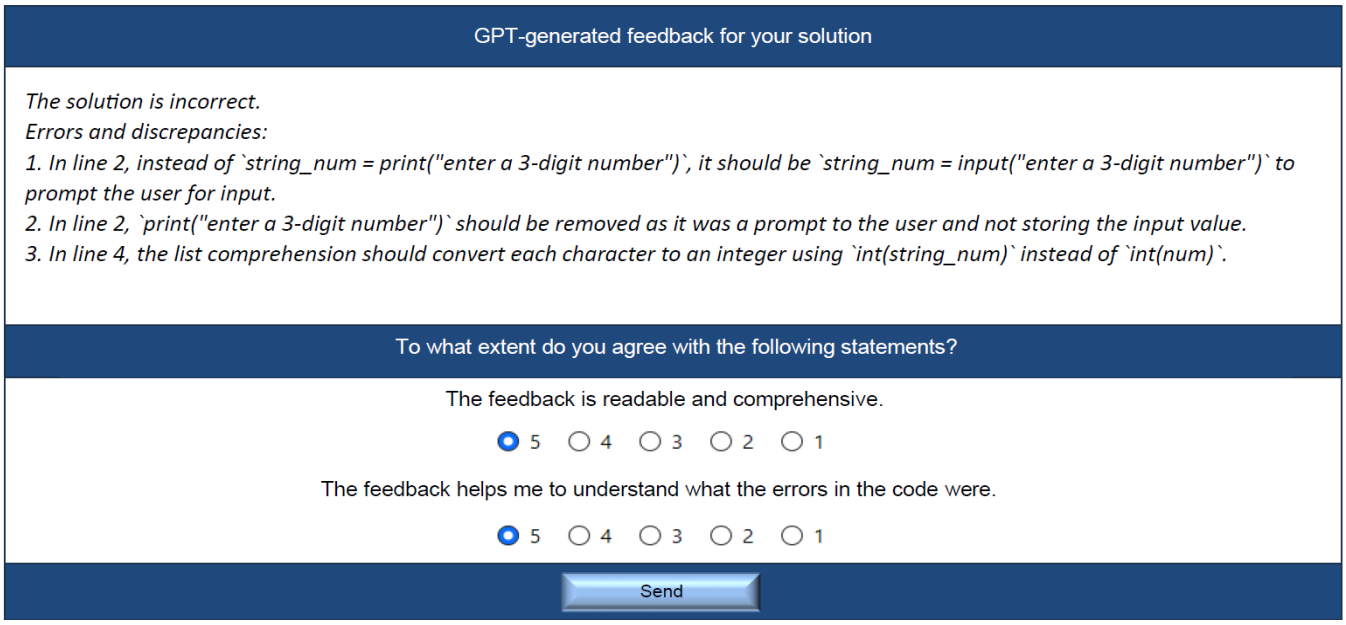

Figure 3: Gipy interface – feedback provided with two queries displayed instantly.

3.5 Gathering Learners' Perceptions

Two research tools were used to collect learners' perceptions of the LLM-generated feedback: 1) Application-embedded queries: two questions presented with incorrect solution feedback to gauge immediate impressions of readability and usability (Figure 3, bottom section), and three additional questions (in rotation) on resubmission, to assess the feedback's perceived value after revising the code; 2) A final questionnaire: At the study's conclusion, participants answered three open-ended questions about the advantages and disadvantages of using GPT for feedback and any other pertinent insights

In the course cycle scheduled for July 2023, participants were recruited from learners already engaged with the ATF system at the onset of the study. Out of 66 learners who voluntarily agreed to participate in our research, only 30 actively utilized the Gipy application. Their activity yielded 532 records of Gipy usage, which were extracted to serve as the dataset for our analysis. Eighteen participants responded to the final questionnaire.

4 RESULTS

4.1 Assessment of LLM-generated Feedback (RQ1)

We first evaluated how correctly the models were able to detect errors in learners' code, utilizing the ATF system as a benchmark. The obtained error detection score for GPT-3.5-turbo (hereinafter referred to as GPT-3.5) is 0.81; for 1220 out of 1500 cases (500 solutions, 3 iterations) the detection was the same as that of the ATF system. Out of the misdetection cases, 18.01% were

False Negative; similarly, 18.88% were False Positive. Further, we compared the correctness indications generated by GPT-3.5 across four distinct error types (Figure 4, coloured bars). Syntax errors (SYE) obtained the highest score, indicating the model's proficiency in syntactic recognition, whereas test-failure errors (TFE) obtained the lowest score, followed by exception errors (EXE), suggesting challenges in recognizing code execution issues. The nonparametric Kruskal Wallis test yielded a statistically significant difference (H(4) = 51.54 ,p<.001). Pairwise multiple comparisons using the nonparametric Dunn's test showed that TFE differed significantly from correct solutions, INE, and SYE, as well as revealing significant differences between SYE and correct solutions (p_{bonf} <.01 in relevant pairwise comparisons).

The error detection score obtained for GPT-4 is 0.90, representing 1356 correct detections out of 1500 cases. The results for the various error types resembled the performance of GPT-3.5 for correct solutions and for SYE and were higher for other error types (Figure 4, grey bars). Significant differences in detection scores emerged for the pairs correct solutions– INS, correct solutions-SYE, as well as for the pairs INE-EXE, INE-TFE and SYE-EXE (H(4) = 51.54 ,p<.001 for the Kruskal Wallis test, p_{bonf} <.05 in relevant pairwise comparisons in the Dunn's test).

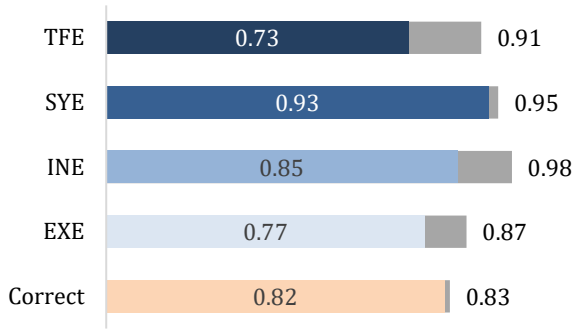

Figure 4: Error detection score across error types (N=500) for GPT-3.5 (coloured bars) and GPT-4 (in grey)

A comparison of the correctness indication across three iterations for each solution in our dataset shows that the feedback generated by GPT-3.5 was consistent in 433 out of 500 solutions, either aligning with or differing from the ATF's indication, yielding a consistency rate of 0.87 (SD=0.34). Higher consistency was demonstrated in detecting INE and SYE as opposed to TFE and EXE (Figure 5, coloured bars). The Kruskal-Wallis test and the post-hoc Dunn's test confirmed that the differences were statistically significant, with H(4)=22.76, p<.001, and p_{bonf} <.01 for the relevant pairwise comparisons. The consistency rate obtained for GPT-4 was 0.94 (SD=0.25), with significantly lower rates for correct solutions compared to all error types in incorrect solutions (H(4) = 41.27, p<.001 in the Kruskal-Wallis test and p_{bonf} <.001 in the Dunn's test).

Within the subset of solutions that were manually evaluated for accuracy (N=158), GPT-3.5 concurred with the error type flagged by the ATF system in 92.4% (148) of cases and GPT-4 concurred in 96% (152) of cases. Assessing the accuracy of the entire feedback provided by GPT-3.5 for each solution yielded a mean rate of 0.78 (SD=0.27). Similar to the error detection and consistency metrics, the accuracy rate differs significantly across error types, with the highest mean for instruction error (INE) and the lowest for TFE (Figure 6, coloured bars). The differences between the rates for INE vs. EXE, SYE, and TFE were found to be significant (H(4)=13.98, p=.003 in the Kruskal-Wallis test, p_{bonf} <.05 in Dunn's test). The mean accuracy rate obtained for the feedback provided by GPT-4 is 0.81 (SD=0.25), with no significant difference in this metric across the various types of errors (Figure 6, grey bars).

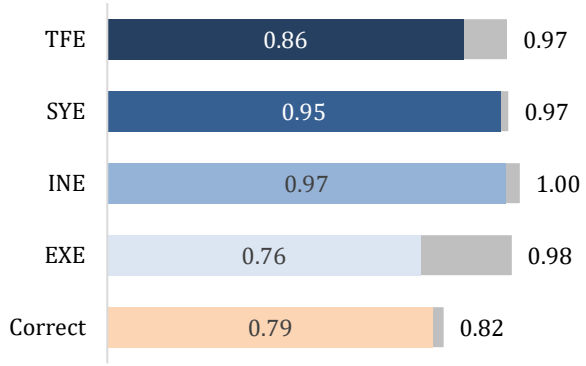

Figure 5: Consistency score across error types (N=500) for GPT-3.5 (coloured bars) and GPT-4 (in grey)

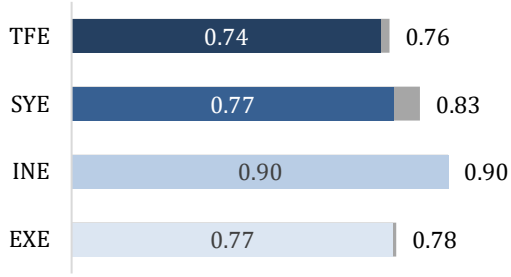

Figure 6: Accuracy rate across error types (N = 158) for GPT-3.5 (coloured bars) and GPT-4 (in grey)

In the majority of cases, both models offered more comprehensive and detailed feedback than the ATF, highlighting additional issues within the code. Specifically, for solutions with errors that prevent test scenario execution (e.g., EXE, INS, or SYN), the GPT-generated feedback highlighted several issues, including conceptual errors (if any), as it addressed the entire code. Conversely, the ATF system's feedback in these situations pinpointed only errors hindering code execution. Figure 7 illustrates an example of such a case in which GPT-3.5 offers

Assignment: 6.4.1
Define a function check_valid_input(letter_guessed, old_letters). Return False if letter_guessed longer than one, not a valid English letter or exists in old_letters

Student's code:

```
def check_valid_input(letter_guessed, old_letters):
    letter_guessed = letter_guessed.lower()
    old_letters = ["$"]
    if len(letter_guessed)==1 and letter_guessed.isalpha()==
True  and letter_guessed in old_letters == False
    old_letters.append(letter_guessed)
    return False
      else:
    return True
```

GPT-3.5-generated feedback:
The solution is incorrect.
1. The variable `old_letters` should not be initialized as `["$"]` inside the function. It should be passed as an argument to the function.
2. The condition `letter_guessed in old_letters == False` is incorrect. It should be `letter_guessed not in old_letters `.
3. The indentation of the code inside the `if` statement is incorrect. It should be indented with four spaces.
4. The function is not returning the correct value when the conditions are met. It should return `True` instead of `False`.

Figure 7: Example of elaborated GPT-generated feedback for incorrect code. The provided feedback refers to the identified error (SYE, third comment) but highlights additional issues in the code.

comprehensive feedback by addressing conceptual errors and not just syntax errors.

Nevertheless, the generated feedback message was completely accurate and pertinent to the code (with an accuracy rate of 1) in only 43.04% (68) of instances for GPT-3.5 and 51.27% (81) of instances for GPT-4. In the remaining cases, the feedback contained at least one incorrect annotation, such as referring to an error that does not exist. Figure 8 shows an example of such feedback, indicating an incorrect function definition, even though it is correct. The example in Figure 8 also shows that GPT-3.5 assesses the implemented functions and remarks on the inclusion of the prohibited component as though it had been directly integrated into the code (e.g., the library function replace() in the example).

The prompt-compliance rate was found to be 0.82 (SD=0.38) for GPT-3.5 and 0.85 (SD=0.36) for GPT-4. Typical examples of non-compliance with the prompt included exposure of a full or partial

Assignment: 3.4.2
Prompt user for a string (the exact prompt is not mandatory); replace all first character occurrences with 'e' (except the first).
Do not use loops.

Student's code:

```
txt = input("please enter string: ")
x = txt + txt[1: ].replace("d", "e")
print(x)
```

GPT-3.5-generated feedback:
The solution is incorrect.
1. The prompt message is missing in the input function.
2. The solution uses the character 'd' as the character to be replaced, instead of the first character occurrences.
3. The solution does not exclude the first character from being replaced.
4. The solution does not follow the requirement of not using loops.

GPT-4-generated feedback:
The solution is incorrect.
1. The implementation incorrectly duplicates the string instead of replacing characters.
2. The replace method is misused to replace all "d" with "e", not the first character as required.
3. The solution does not meet the requirement to replace the first character occurrences with 'e' (except the first).
4.The variable name "x" is not ideal. It is not descriptive and does not convey the purpose or content of the variable.

Figure 8: Inaccurate Feedback. GPT-3.5 feedback refers to a non-existing error (comment 1) and a library function (comment 4). GPT-4 feedback addresses naming conventions (comment 4)

solution or suggestions for code improvements that were not explicitly required by the assignment and sometimes not suitable to learners' expected knowledge level. GPT-4 tended to point out minor issues, such as name conventions and style (Figure 8, bottom).

4.2 Learners' Experience with the LLM-generated Feedback (RQ2, RQ3)

In the field experiment, participants used the Gipy application to make 532 submissions, receiving feedback generated by GPT-3.5. Of these, 34.4% (183 submissions) were flagged as erroneous, prompting learners to share their initial impressions of the feedback provided. Table 3 summarizes learners' responses to two

questions asked during each submission, as well as to three additional questions posed upon resubmission of the same assignment. Fewer responses to these additional questions result from both their rotational presentation and the fact that participants did not always resubmit. To account for individual rating biases, we first averaged the ratings of each learner and then calculated the overall average.

Feedback readability (Statement 1) received higher scores, while comprehension of key messages, especially error indications, was rated significantly lower (Wilcoxon test, Z=6.29, p<.001). Feedback accuracy and helpfulness in solving assignments (Statements 3, 4) received moderate ratings as well. Moreover, learners did not prefer Gipy's feedback over feedback provided by ATF (Statement 5). Analysing learners' responses to open-ended questions provided additional insights into their perspectives on the LLM-generated feedback. When asked about the advantages for learning programming, many learners noted that the LLM-generated feedback was able to identify a wider range of errors, surpassing ATF in this aspect. For instance, one participant commented, "It can catch more errors and bugs, including borderline cases that might be missed in manually created tests." Learners also valued the context-sensitive, detailed, and understandable explanations provided by the LLM-generated feedback, in contrast to the ATF.

Table 3: Average learners' agreement with statements on Gipy feedback (Likert scale, range 1-5)

Statement	N	Mean	SD
Q1: The feedback is readable and comprehensive.	183	3.83	1.14
Q2: The feedback helps me to understand what the errors in the code are.	183	2.98	1.51
Q3: The provided feedback was accurate and matched the errors in the code.	37	3.08	1.44
Q4: With the provided feedback, it was clear to me how to improve the code.	52	2.98	1.45
Q5: The feedback provided by Gipy was more valuable than that from the ATF system for the same code.	40	3.06	1.31

Learners' responses particularly highlighted the feedback's tailored approach of addressing specific issues within each code. For instance, one learner noted, "It addresses the code in a 'personal' manner, referring to specific problems". Another mentioned, "It aims to explain the code's issues, unlike ATF, which only indicates the output for given input". Additionally, learners found value in the guidance provided on coding best practices and standards. One respondent stated, "It helps improve code writing according to acceptable standards".

In response to the question about the disadvantages or negative aspects of LLM-generated feedback in a programming course, a recurrent theme was its occasional unreliability (in particular for complex code), leading to potential misguidance. For example, one respondent wrote: "It is not always 100% reliable and might be unpredictable. In rare cases, it can be misleading". Learners also pointed out its inconsistency, as different feedback can be given for identical code. Some noted that the feedback, though containing correct statements, can also mix in incorrect ones, necessitating reading the feedback critically: "Gipy often provided one accurate comment (occasionally pivotal for completing the exercise) accompanied by 4-5 inaccurate ones. It's essential to filter through the entire feedback, as often many of the comments are off the mark". Some learners mentioned that GPT occasionally revealed solutions deemed counterproductive to the learning process.

Learners suggested that Gipy should be refined to enhance the quality of LLM-generated feedback, seeing it as a potential supplement to ATF. They advised integrating Gipy in a way that addresses its inaccuracies and occasional "hallucinations". A learner described a useful strategy: "First, I check the code with ATF for correctness. Then, for errors, I turn to Gipy for detailed explanations." Another highlighted its value in self-learning: "There's no need for a teacher or programmer nearby; GPT can fill that role. With some improvement, it will be a great tool."

5 DISCUSSION

Our evaluation of the feedback generated by the two GPT versions compared to the ATF feedback yielded both strengths and limitations. On the positive side, the models were quite effective in identifying errors, in line with previous studies [18, 22]. Analysis of the LLM-generated feedback suggests that it seems to address the main gaps previously noted in test-based feedback [34]: The LLM-generated feedback was tailored to the specific submission in that it referred to the errors found throughout the code and provided detailed information about each of them. Particularly noteworthy were cases in which the code execution failed. While the ATF merely pointed out errors (due to its inability to run test scenarios), the GPTs offered a comprehensive explanation of the issues. Study participants found this strength, also mentioned by [35], to be invaluable in that it potentially spared them the frustration of "searching in the dark" for elusive errors and reduced the number of fix-and-resubmit iterations.

At the same time, the results raise concerns about the quality and reliability of LLM-generated feedback. More than half of the feedback instances generated by GPT-3.5 (or just about half for GPT-4) contained at least one inaccurate annotation. This low observed accuracy can potentially undermine learners' trust. In particular, instances in which non-existent errors—termed as "hallucination" by [3]—were highlighted may cause frustration among learners attempting to resolve these "imaginary" problems and could reduce confidence, particularly in novice learners [22,

40]. Another concern is the potentially high rate of False Positive (FP) misdetections for conceptual issues resulting from the relatively low detection score for test-failed and execution errors. This issue is less pronounced in the newer model but it still exists. The mistaken consideration of incorrect code as correct may lead novice learners to develop misconceptions about the solution [42]. Moreover, according to [27] learners are less likely to detect FPs from LLM-generated feedback, thus harming their learning. In the context of independent learning such as MOOC, this concern becomes more pertinent.

A pedagogical issue also arises from the inability to fully enforce the directives of the prompt in that on occasion (more frequently with GPT-3.5) a solution to the assignment was provided within the feedback. While providing a sample solution can be conducive to learning, such a solution should be suggested on demand, enabling learners to decide whether or not to view it [14, 45]. Crafting a meticulous prompt while balancing between offering explanations and potentially hindering the learning process is a complex endeavour. Therefore, instructors should be aware of a potential "loss of control" to a certain extent, inherent in utilizing LLMs to generate feedback in terms of adhering to desired pedagogical principles.

When tested in practice, the learners' instant impressions of the LLM-generated feedback were inconclusive and resonated with our findings from the offline assessment for GPT-3.5. The readability of the feedback and its clarity to learners largely demonstrate the core capability of LLMs in generating human-like text [39]. However, learners' experience with frequent inaccuracies via the Gipy application may influence their views on its usefulness and contribution [27]. Comparing the Gipy feedback to that of a trusted source, specifically the ATF system, may have fostered a less tolerant attitude toward its inaccuracies, thereby diminishing the perceived pedagogical value of the GPT feedback. To obtain an authentic response, we refrained from informing learners about possible correctness and accuracy issues of the feedback. Raising learners' awareness regarding potential inaccuracies, as suggested by [28], may assist them in navigating these deficiencies and may cultivate a more favourable understanding of the benefits of LLM-generated feedback.

In response to the emergence of the advanced GPT-4 model during our study, we sought to assess whether and how it improves feedback generation for code assignments. While not statistically tested, our findings echo the study of [29], suggesting enhanced error detection and consistency, especially for error types for which GPT-3.5 yields lower scores. These enhancements were less noticeable for correct solutions. Reviewing feedback content in False Negative cases revealed that GPT-4 is more likely than GPT-3.5 to note minor errors even when the code meets the assignment requirements. This propensity may contribute to the similar accuracy rates observed for both models, albeit with distinct types of inaccuracies. Optimizing the prompt to specifically avoid such minor comments could highlight the expected benefits of the advanced model more distinctly.

6 LIMITATIONS

The self-paced format, which enables learners to begin and complete the course at varying times, combined with the constrained duration of the research, resulted in a modest number of participants. Moreover, learners' engagement fell short of our expectations, and they did not often upload their solutions to both Gipy and the ATF system. In a follow-up study, we intend to extend the scope of our experiment across the full course duration with the aim of allowing more learners to participate so as to be more representative of the "average learner" and obtain more evidence-based results.

The language of instruction in the course is not English. Nevertheless, we opted to provide feedback in English so as to avoid translation that could distort the text's accuracy. This choice may have affected learners' understanding and perception of the feedback's value, particularly compared to the feedback provided by the ATF system in the course language. With advancements in the multilingual capabilities of LLMs, we anticipate being able to offer feedback in the course language for more accurate learner comprehension.

Our qualitative analysis employed a single expert, potentially limiting the robustness and breadth of our findings.

The swift advancement of LLMs constitutes a significant limitation by casting doubt on the long-term relevance of our findings. The emergence of GPT-4 with its enhanced feedback capabilities during the study suggests that further versions and new models may substantially outperform current ones. Therefore, ongoing assessment of the capabilities and feedback quality of emerging versions is imperative.

7 CONCLUSIONS AND FURTHER RESEARCH

Providing effective feedback on code assignments in large-scale programming courses poses a significant challenge. While the commonly used test-based feedback is reliable and precise, it is often inadequate. In this study, we explore the potential of LLM-generated feedback to address this need. In a MOOC for programming, we assessed the feedback generated by two LLMs, GPT-3.5-turbo and GPT-4, and analysed learners' perceptions of the feedback provided by GPT-3.5-turbo through a dedicated application. Both models demonstrated effective error detection, with GPT-4 surpassing its predecessor. The generated feedback often complemented gaps in the ATF. Yet concerns arose about the accuracy of the feedback, casting doubts on the advisability of relying solely on LLMs, especially GPT-3.5, for code assignment feedback. This concern is particularly pertinent for novices. The learners' perceptions regarding the benefits and drawbacks of LLM-generated feedback were not conclusive. Nevertheless, given the accessibility of generative AI tools such as ChatGPT, it is plausible that programming learners are already using them, for better or for worse.

Therefore, considering both the potential value and the inherent drawbacks of LLM-generated feedback, a viable approach would be to integrate this feedback with reliable ATF feedback. An interface application such as Gipy may provide a

more refined pedagogical framework than direct chat with LLM tools, thus optimizing utilization. Our experience shows that with minimal resources, a cost-effective and user-friendly tool tailored specifically for a particular course can be developed. By understanding LLM limitations and critically comparing feedback modalities, learners can effectively use LLM feedback capabilities to enhance their learning experience. Furthermore, comparing and critically examining the two feedback modalities may further enhance learners' educational experience.

In future research we seek to investigate the impact of using LLM-generated feedback on learning outcomes and the acquisition of programming skills, in particular in large-scale and independent learning contexts. This research direction is notably significant in the context of balancing effective learning through GenAI tools against potential over-dependence. Indeed, it may lead to new pedagogical considerations, such as incorporating innovative types of assignments. Furthermore, to provide learners with up-to-date assistance, we suggest comparing the performance of newer models in generating feedback, focusing on models specifically trained on programming data, such as advanced versions of GitHub's Copilot X.

ACKNOWLEDGMENTS

We express our gratitude to the Azrieli Foundation for awarding us a generous Azrieli Fellowship, which facilitated this study. Further, we would also like to thank the Cyber Education Center for enabling us to conduct the research.

REFERENCES

[1] Balse, R., Kumar, V., Prasad, P. and Warriem, J.M. 2023. Evaluating the Quality of LLM-Generated Explanations for Logical Errors in CS1 Student Programs. *ACM International Conference Proceeding Series*. (Dec. 2023), 49–54. DOI:https://doi.org/10.1145/3627217.3627233.

[2] Balse, R., Valaboju, B., Singhal, S., Prasad, P. and Madathil Warriem, J. 2023. Investigating the Potential of GPT-3 in Providing Feedback for Programming Assessments. *The 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (2023), 292–298.

[3] Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W., Do, Q. V., Xu, Y. and Fung, P. 2023. A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity. *arXiv preprint arXiv:2302.04023*. (2023).

[4] Becker, B.A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J. and Santos, E.A. 2023. Programming Is Hard - or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation. *SIGCSE 2023 - Proceedings of the 54th ACM Technical Symposium on Computer Science Education*. 1, July (2023), 500–506. DOI:https://doi.org/10.1145/3545945.3569759.

[5] Cai, Y.Z. and Tsai, M.H. 2019. Improving Programming Education Quality with Automatic Grading System. *International Conference on Innovative Technologies and Learning* (Dec. 2019), 207–215.

[6] Combéfis, S. 2022. Automated Code Assessment for Education: Review , Classification and Perspectives on Techniques and Tools. *Software 2022*. 1, (2022), 3–30. DOI:https://doi.org/https://doi.org/10.3390/ software1010002.

[7] Denny, P., Khosravi, H., Hellas, A., Leinonen, J. and Sarsa, S. 2023. Can We Trust AI-Generated Educational Content? Comparative Analysis of Human and AI-Generated Learning Resources. (2023), 1–15.

[8] Denny, P., Kumar, V. and Giacaman, N. 2023. Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language. *Proceedings of the 54th ACM Technical Symposium on Computer Science, 2023* (Mar. 2023), 1136–1142.

[9] Denny, P., Prather, J., Becker, B.A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B.N., Santos, E.A. and Sarsa, S. 2023. Computing Education in the Era of Generative AI. *arXiv preprint arXiv:2306.02608*. (2023).

[10] Denny, P., Prather, J., Becker, B.A., Mooney, C., Homer, J., Albrecht, Z. and Powell, G. 2021. On Designing Programming Error Messages for Novices: Readability and its Constituent Factors. *2021 CHI Conference on Human Factors in Computing Systems* (2021), 1–15.

[11] Finnie-Ansley, J., Denny, P., Becker, B.A., Luxton-Reilly, A. and Prather, J. 2022. The robots are coming: Exploring the implications of openai codex on introductory programming. *Proceedings of the 24th Australasian Computing Education Conference, 2022* (Feb. 2022), 10–19.

[12] Finnie-Ansley, J., Denny, P., Luxton-Reilly, A., Santos, E.A., Prather, J. and Becker, B.A. 2023. My AI Wants to Know if This Will Be on the Exam: Testing OpenAI's Codex on CS2 Programming Exercises. *Proceedings of the 25th Australasian Computing Education Conference, 2023* (Jan. 2023), 97–104.

[13] Gabbay, H. and Cohen, A. 2022. Exploring learners ' data of automated feedback system in online programming course: what can be learned?

[14] Gabbay, H. and Cohen, A. 2023. Unfolding Learners' Response to Different Versions of Automated Feedback in a MOOC for Programming – A Sequence Analysis Approach. *EDM 2023 - Proceedings of the 16h International Conference on Educational Data Mining* (2023), 160–170.

[15] Gordillo, A. 2019. Effect of an Instructor-Centered Tool for Automatic Assessment of Programming Assignments on Students' Perceptions and Performance. *Sustainability*. 11, 20 (Oct. 2019), 5568. DOI:https://doi.org/10.3390/su11205568.

[16] Hackl, V., Müller, A.E., Granitzer, M. and Sailer, M. 2023. Is GPT-4 a reliable rater? Evaluating Consistency in GPT-4 Text Ratings. *arXiv preprint arXiv:2308.02575 (2023)*. (2023), 1–14.

[17] Hao, Q., Smith IV, D.H., Ding, L., Ko, A., Ottaway, C., Wilson, J., Arakawa, K.H., Turcan, A., Poehlman, T. and Greer, T. 2022. Towards understanding the effective design of automated formative feedback for programming assignments. *Computer Science Education*. 32, 1 (Jan. 2022), 105–127. DOI:https://doi.org/10.1080/08993408.2020.1860408.

[18] Hellas, A., Leinonen, J., Sarsa, S., Koutcheme, C., Kujanpää, L. and Sorva, J. 2023. Exploring the Responses of Large Language Models to Beginner Programmers' Help Requests. *arXiv:2306.05715 [cs.CY]*. (2023). DOI:https://doi.org/10.1145/3568813.3600139.

[19] Jukiewicz, M. 2023. The Future of Grading Programming Assignments in Education: The Role of ChatGPT in Automating the Assessment and Feedback Process. (2023). DOI:https://doi.org/10.13140/RG.2.2.22103.85924.

[20] Kanti Karmaker, S. and Feng, D. 2023. TELeR: A General Taxonomy of LLM Prompts for Benchmarking Complex Tasks. *arXiv preprint arXiv:2305.11430 (2023)*. (2023).

[21] Keuning, H., Jeuring, J. and Heeren, B. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*. 19, 1 (2018), 1–43. DOI:https://doi.org/10.1145/3231711.

[22] Kiesler, N., Lohr, D. and Keuning, H. 2023. Exploring the Potential of Large Language Models to Generate Formative Programming Feedback. *Proceedings of the 2023 IEEE ASEE Frontiers in Education Conference* (2023), 1–5.

[23] Kiesler, N. and Schiffner, D. 2023. Large Language Models in Introductory Programming Education: ChatGPT 's Performance and Implications for Assessments. August (2023). DOI:https://doi.org/10.48550/arXiv.2308.08572.

[24] Koutcheme, C., Sarsa, S., Leinonen, J., Hellas, A. and Denny, P. 2023. Automated Program Repair Using Generative Models for Code Infilling. *International Conference on Artificial Intelligence in Education* (2023), 798–803.

[25] Leinonen, J., Denny, P., Macneil, S., Sarsa, S., Bernstein, S., Kim, J., Tran, A. and Hellas, A. 2023. Comparing code explanations created by students and large language models. *arXiv preprint arXiv:2304.03938, 2023*. (Jun. 2023). DOI:https://doi.org/10.1145/3587102.3588785.

[26] Leinonen, J., Hellas, A., Sarsa, S., Reeves, B., Denny, P., Prather, J. and Becker, B.A. 2023. Using large language models to enhance programming error messages. *Proceedings of the 54th ACM Technical Symposium on Computer Science, 2023* (Mar. 2023), 563–569.

[27] Li, T.W., Hsu, S., Fowler, M., Zhang, Z., Zilles, C. and Karahalios, K. 2023. Am I Wrong, or Is the Autograder Wrong? Effects of AI Grading Mistakes on Learning. *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1 (ICER '23)*. 1, 23 (2023). DOI:https://doi.org/10.1145/3568813.3600124.

[28] Liffiton, M., Sheese, B., Savelka, J. and Denny, P. 2023. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. *arXiv preprint arXiv:2308.06921 (2023)*. (2023).

[29] Macneil, S., Denny, P., Tran, A., Leinonen, J., Bernstein, S., Hellas, A., Sarsa, S. and Kim, J. 2024. Decoding Logic Errors: A Comparative Study on Bug Detection by Students and Large Language Models. *Proceedings of Australasian Computing Education Conference (ACE 2024)*. 1, (2024).

[30] Macneil, S., Leinonen, J., Bernstein, S., Becker, B.A., Kim, J., Denny, P., Wermelinger, M., Hellas, A., Tran, A., Sarsa, S., Prather, J. and Kumar, V. 2023. The Implications of Large Language Models for CS Teachers and Students. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (Vol. 2)*. (2023).

[31] Maier, U. and Klotz, C. 2022. Personalized feedback in digital learning environments: Classification framework and literature review. *Computers and Education: Artificial Intelligence*100080 ,(2022) ,3 .

[32] Marwan, S. and Price, T.W. 2022. iSnap : Evolution and Evaluation of a Data-Driven Hint System for Block-based Programming. *IEEE Transactions on Learning Technologies*. (2022), 1–15. DOI:https://doi.org/10.1109/TLT.2022.3223577.

[33] Narciss, S. 2013. Designing and evaluating tutoring feedback strategies for digital learning environments on the basis of the interactive tutoring feedback model. *Digital Education Review*. 23, 1 (2013), 7–26. DOI:https://doi.org/10.1344/der.2013.23.7-26.

[34] Paiva, J.C., Leal, P., Figueira, Á., Leal, J.P. and Figueira, A. 2022. Automated Assessment in Computer Science Education: A State-of-the-Art Review. *ACM Transactions on Computing Education (TOCE)*. 22, 3 (Jun. 2022), 1–40. DOI:https://doi.org/10.1145/3513140.

[35] Pankiewicz, M. and Baker, R. 2023. Large Language Models (GPT) for automating feedback on programming assignments. *Proceedings of the 31st International Conference on Computers in Education (ICCE)* (2023).

[36] Pettit, R., Homer, J.D., Holcomb, K.M., Simone, N. and Mengel, S.A. 2015. Are automated assessment tools helpful in programming courses? *ASEE Annual Conference and Exposition, Conference Proceedings*. 122nd ASEE, 122nd ASEE Annual Conference and Exposition: Making Value for Society (2015). DOI:https://doi.org/10.18260/p.23569.

[37] Phung, T., Cambronero, J., Gulwani, S., Kohn, T., Majumdar, R., Singla, A. and Soares, G. 2023. Generating High-Precision Feedback for Programming Syntax Errors using Large Language Models. *arXiv:2302.04662 [cs.PL]*. (2023).

[38] Pieterse, V. 2013. Automated Assessment of Programming Assignments. *3rd Computer Science Education Research Conference on Computer Science Education Research*. 3, April (2013), 45–56. DOI:https://doi.org/http://dx.doi.org/10.1145/1559755.1559763.

[39] Prather, J. et al. 2023. Transformed by Transformers: Navigating the AI Coding Revolution for Computing Education. An ITiCSE Working Group Conducted by Humans. *2023 Conference on Innovation and Technology in Computer Science Education V. 2* (2023), 561–562.

[40] Prather, J., Reeves, B.N., Denny, P., Becker, B.A., Leinonen, J., Luxton-Reilly, A., Powell, G., Finnie-Ansley, J. and Santos, E.A. 2023. "It's Weird That it Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *arXiv preprint arXiv:2306.02608*. (Apr. 2023).

[41] Si, C., Gan, Z., Yang, Z., Wang, S., Wang, J., Boyd-Graber, J. and Wang, L. 2022. Prompting gpt-3 to be reliable. *arXiv preprint*. (2022).

[42] Vaithilingam, P., Zhang, T. and Glassman, E.L. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. *Conference on Human Factors in Computing Systems - Proceedings*. (2022). DOI:https://doi.org/10.1145/3491101.3519665.

[43] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J. and Schmidt, D.C. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. (Feb. 2023).

[44] Zhang, J., Cambronero, J., Redmond, M., Le, V., Piskac, R., Soares, G. and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. *arXiv preprint arXiv:2209.14876*. (2022).

[45] Zhou, Y., Andres-Bray, J.M., Hutt, S., Ostrow, K. and Baker, R.S. 2021. A Comparison of Hints vs. Scaffolding in a MOOC with Adult Learners. *Artificial Intelligence in Education: 22nd International Conference, AIED 2021, Proceedings, Part II* (Utrecht, The Netherlands, Jun. 2021), 427–432.